



Universidad de Cuenca

Facultad de Ingeniería
Ingeniería en Telecomunicaciones

Microprocesadores, Microcontroladores y Sistemas Embebidos

Kenneth S. Palacio Baus
kenneth.palacio@ucuenca.edu.ec

Jaime Patricio Chiqui Chiqui
jpatricio.chiquic@ucuenca.edu.ec

Práctica 5

PIC18F4550 - Manejo de pantalla LCD 16×2 y Conversor Analógico Digital ADC

- **Valoración:** 10 puntos.
- **Tipo de Trabajo:** Trabajo práctico e informe y defensa individual.
- **Objetivos:** Mediante la presente práctica el estudiante aprenderá a:
 - Visualizar información de carácter alfa-numérico en una pantalla LCD de 16×2 caracteres mediante la comunicación con el controlador Hitachi HD44780.
 - Manejar el módulo conversor analógico digital (ADC) integrado en el PIC18F4550.
 - Visualizar el resultado de la conversión ADC de 10 bits en un display LCD.
 - Integrar un sistema de ingreso de datos mediante teclado matricial con visualización inteligente.
 - Realizar tareas básicas de toma de decisiones basadas en una medición analógica.
- **Recursos:** MPLAB X IDE, compilador XC8 C y hoja de datos del PIC18F4550.
- **Instrucciones:**
 - Envíe su trabajo mediante la plataforma e-nova.
 - Nombre del archivo: ApellidoNombre_Pract05_MicroCon_M26.pdf
 - Si el envío no cumple con el nombre de archivo y fecha, no recibirá calificación.

1. Materiales y Equipos

Para el desarrollo de la presente práctica, se utilizaron los siguientes recursos y componentes electrónicos:

- **Software:** MPLAB X IDE v5.50, compilador XC8 C.
- **Microcontrolador:** PIC18F4550 de Microchip con cristal de cuarzo 20 MHz.
- **Pantalla LCD:** Display LCD 16×2 caracteres con controlador Hitachi HD44780.
- **Teclado Matricial:** 16 teclas en configuración 4×4 (4 filas, 4 columnas).
- **Potenciómetro:** R11 de 100 kΩ para entrada analógica al canal AN0.
- **Resistencias:**
 - 1 resistencia de 1 kΩ (limitadora del LED en RD0).
 - 4 resistencias de 10 kΩ (pull-up para columnas del teclado).
 - 1 resistencia de 220 Ω (contraste LCD, VO).
 - 1 potenciómetro de 1 kΩ (retroiluminación LCD, Vo del potenciómetro de contraste).
- **Oscilador:** Cristal de cuarzo de 20 MHz con 2 capacitores cerámicos de 22 pF.
- **Programador:** *PICkit 3* con su respectivo cable USB y conector espadín de 6 pines.
- **LED:** LED indicador conectado en RD0.
- **Otros:** Protoboard, cables de conexión calibre 22 (o jumpers).

2. Marco Teórico

2.1. Pantalla LCD 16×2 con controlador HD44780

Una pantalla LCD (*Liquid Crystal Display*) de 16×2 es un módulo de visualización alfa-numérico capaz de mostrar hasta 32 caracteres distribuidos en 2 filas de 16 columnas cada una. Su controlador integrado, el Hitachi HD44780, gestiona internamente la memoria de visualización (DDRAM), el generador de caracteres (CGROM) y la interfaz de comunicación con el microcontrolador.

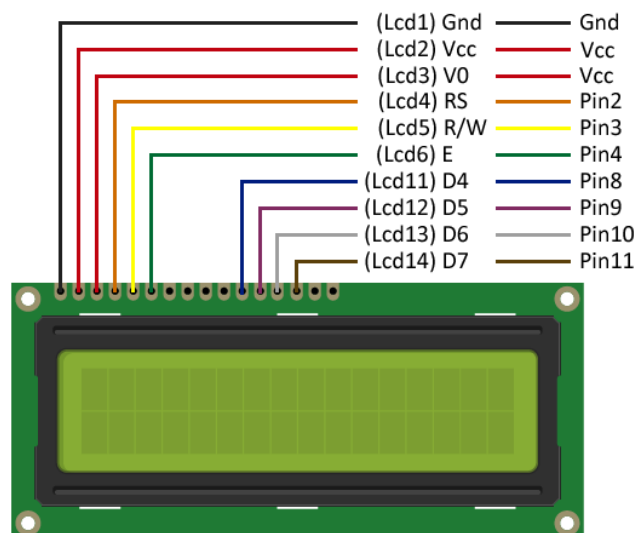


Figura 1: Pantalla LCD 16×2 con controlador HD44780

2.1.1. Pines del LCD

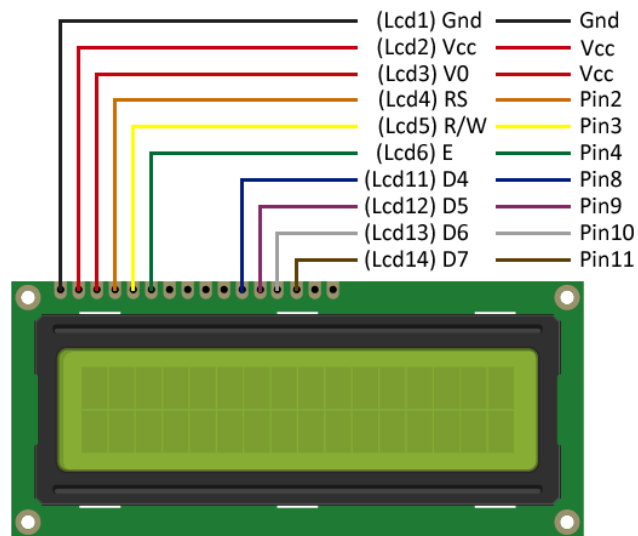


Figura 2: Pines pantalla LCD 16×2 con controlador HD44780

2.1.2. Modo de Comunicación 4 Bits

En lugar de usar los 8 pines de datos (D0–D7), el modo de 4 bits utiliza únicamente D4–D7, reduciendo el número de pines requeridos. Cada byte se transmite en dos ciclos: primero el nibble alto (D7–D4) y luego el nibble bajo (D3–D0), ambos con un pulso EN entre medias.

PIN DESCRIPTIONS

Pin	Symbol	Level	Function
1	GND	0V	GND
2	V _{DD}	5V	Power supply for logic
3	V ₀	--	No connection
4	RS	H/L	H : Data L : Instruction code
5	R/W	H/L	H : Read L : Write
6	E	H, H $\Rightarrow$ L	Enable signal Read data when E is high Write data at falling edge of E
7	DB0	H/L	Data bus
8	DB1	H/L	
9	DB2	H/L	
10	DB3	H/L	
11	DB4	H/L	
12	DB5	H/L	
13	DB6	H/L	
14	DB7	H/L	
15	LEDA	5V	Power supply for LED backlight
16	LEDK	0V	

Figura 3: Comunicación LCD en modo 4 bits: secuencia de nibbles

2.1.3. Comandos Principales del HD44780

Cuadro 1: Comandos principales del controlador HD44780

Comando (Hex)	Descripción
0x01	Limpiar pantalla y regresar cursor al inicio
0x0C	Display ON, cursor OFF, sin parpadeo
0x06	Modo de entrada: cursor avanza a la derecha
0x28	Función: 4 bits, 2 líneas, caracteres 5×8
0x80	Cursor en inicio de primera línea (DDRAM = 0x00)
0xC0	Cursor en inicio de segunda línea (DDRAM = 0x40)
0x33	Secuencia de inicialización (forzar modo 8 bits)
0x32	Secuencia de inicialización (pasar a modo 4 bits)

2.2. Módulo ADC del PIC18F4550

El PIC18F4550 incorpora un convertidor analógico-digital (ADC) de 10 bits con hasta 13 canales de entrada (AN0–AN12). Convierte un voltaje analógico entre 0 V y V_{DD} (5 V) en un número entero de 0 a 1023.

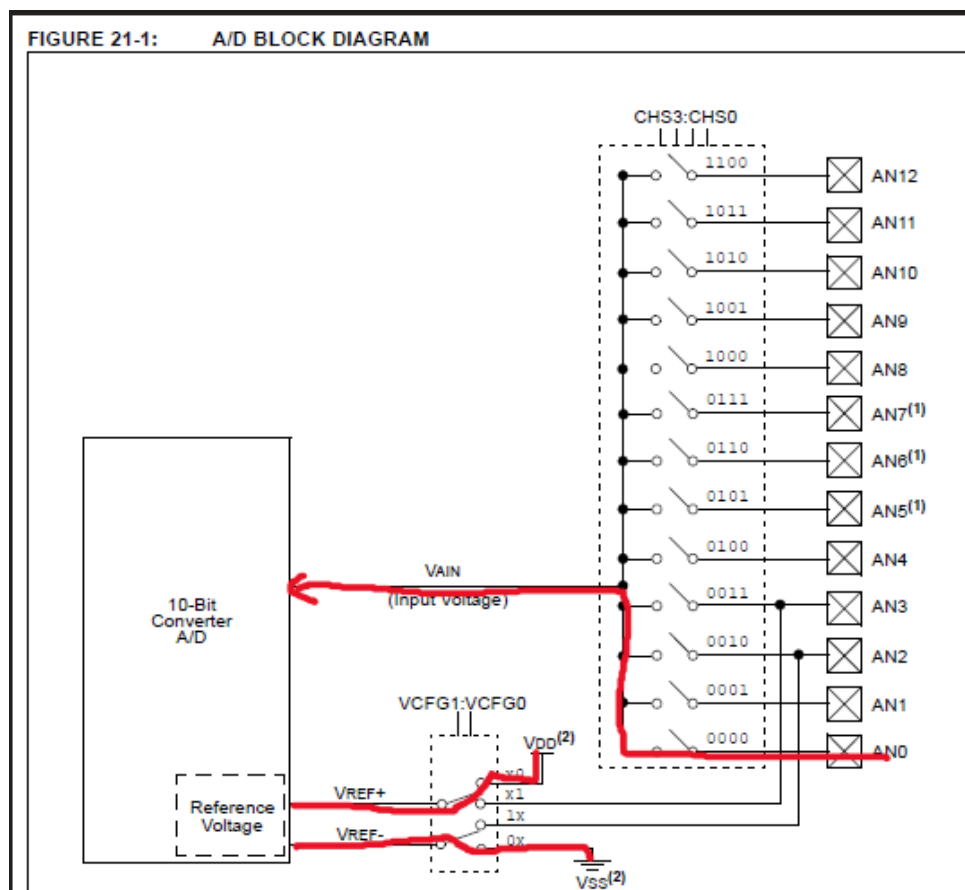


Figura 4: Diagrama de bloques del módulo ADC del PIC18F4550

2.2.1. Registros de Control del ADC

- **ADCON0:** Controla el canal seleccionado (CHS3:CHS0), el bit de inicio de conversión (GO/DONE) y la habilitación del módulo (ADON).

- **ADCON1:** Configura qué pines son analógicos y cuáles digitales. 0x0E configura únicamente AN0 como analógico.
- **ADCON2:** Define la justificación del resultado (ADFM), el tiempo de adquisición (ACQT) y el reloj del ADC (ADCS).
- **ADRESH/ADRESL:** Registros de 8 bits donde se almacena el resultado de 10 bits. Con justificación derecha (ADFM=1): los 2 bits más significativos están en ADRESH y los 8 restantes en ADRESL.

REGISTER 21-1: ADCON0: A/D CONTROL REGISTER 0							
U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7-6	Unimplemented: Read as '0'
bit 5-2	CHS3:CHS0: Analog Channel Select bits 0000 = Channel 0 (AN0) 0001 = Channel 1 (AN1) 0010 = Channel 2 (AN2) 0011 = Channel 3 (AN3) 0100 = Channel 4 (AN4) 0101 = Channel 5 (AN5) ^(1,2) 0110 = Channel 6 (AN6) ^(1,2) 0111 = Channel 7 (AN7) ^(1,2) 1000 = Channel 8 (AN8) 1001 = Channel 9 (AN9) 1010 = Channel 10 (AN10) 1011 = Channel 11 (AN11) 1100 = Channel 12 (AN12) 1101 = Unimplemented ⁽²⁾ 1110 = Unimplemented ⁽²⁾ 1111 = Unimplemented ⁽²⁾
bit 1	GO/DONE: A/D Conversion Status bit <u>When ADON = 1:</u> 1 = A/D conversion in progress 0 = A/D Idle
bit 0	ADON: A/D On bit 1 = A/D converter module is enabled 0 = A/D converter module is disabled

Figura 5: ADCON0 del módulo ADC del PIC18F4550

PCFG3: PCFG0	AN12	AN11	AN10	AN9	AN8	AN7 ⁽²⁾	AN6 ⁽²⁾	AN5 ⁽²⁾	AN4	AN3	AN2	AN1	AN0
0000 ⁽¹⁾	A	A	A	A	A	A	A	A	A	A	A	A	A
0001	A	A	A	A	A	A	A	A	A	A	A	A	A
0010	A	A	A	A	A	A	A	A	A	A	A	A	A
0011	D	A	A	A	A	A	A	A	A	A	A	A	A
0100	D	D	A	A	A	A	A	A	A	A	A	A	A
0101	D	D	D	A	A	A	A	A	A	A	A	A	A
0110	D	D	D	D	A	A	A	A	A	A	A	A	A
0111 ⁽¹⁾	D	D	D	D	D	A	A	A	A	A	A	A	A
1000	D	D	D	D	D	D	A	A	A	A	A	A	A
1001	D	D	D	D	D	D	D	A	A	A	A	A	A
1010	D	D	D	D	D	D	D	D	A	A	A	A	A
1011	D	D	D	D	D	D	D	D	D	A	A	A	A
1100	D	D	D	D	D	D	D	D	D	D	A	A	A
1101	D	D	D	D	D	D	D	D	D	D	D	A	A
1110	D	D	D	D	D	D	D	D	D	D	D	D	A
1111	D	D	D	D	D	D	D	D	D	D	D	D	D

A = Analog input
D = Digital I/O

Figura 6: ACTIVAR SOLO AN0 COMO ANALOGICO Y LOS DEMAS COMO DIGITAL

Figura 7: ADCON2 del módulo ADC del PIC18F4550

2.2.2. Fórmula de Conversión

El valor digital obtenido por el ADC se calcula como:

$$ADC_{valor} = \frac{V_{entrada} \times 1023}{V_{DD}} = \frac{V_{entrada} \times 1023}{5}$$

Para $V_{entrada} = 2,5 \text{ V}$: $ADC_{valor} = \frac{2,5 \times 1023}{5} = 511$

2.2.3. Configuración del Reloj ADC (ADCS)

Con $F_{osc} = 48 \text{ MHz}$, se selecciona $ADCS = 110$ ($F_{osc}/64$):

$$T_{AD} = \frac{64}{48 \text{ MHz}} = 1,33 \mu\text{s} \quad (\text{mínimo requerido: } 0,7 \mu\text{s})$$

TABLE 21-1: TAD vs. DEVICE OPERATING FREQUENCIES

AD Clock Source (TAD)		Assumes TAD Min. = 0.8 μs
Operation	ADCS2:ADCS0	Maximum Fosc
2 TOSC	000	2.50 MHz
4 TOSC	100	5.00 MHz
8 TOSC	001	10.00 MHz
16 TOSC	101	20.00 MHz
32 TOSC	010	40.00 MHz
64 TOSC	110	48.00 MHz
RC ⁽²⁾	x11	1.00 MHz ⁽¹⁾

Figura 8: TAD vs device operating frequencies del PIC18F4550

2.3. Timer0 para Muestreo Periódico del ADC

El TMR0 en modo de 16 bits con prescaler 1:256 genera interrupciones cada 200 ms:

- $T_{cy} = \frac{4}{48 \text{ MHz}} = 83,33 \text{ ns}$
- $T_{tick} = 83,33 \text{ ns} \times 256 = 21,33 \mu\text{s}$
- Cuentas: $N = \frac{200 \text{ ms}}{21,33 \mu\text{s}} = 9375$
- Precarga: $65536 - 9375 = 56161 = 0\text{xDB61}$
- Por lo tanto: $TMR0H = 0\text{xDB}$, $TMR0L = 0\text{x61}$

3. Diseño del Sistema Microcontrolado

3.1. Plataforma de Hardware

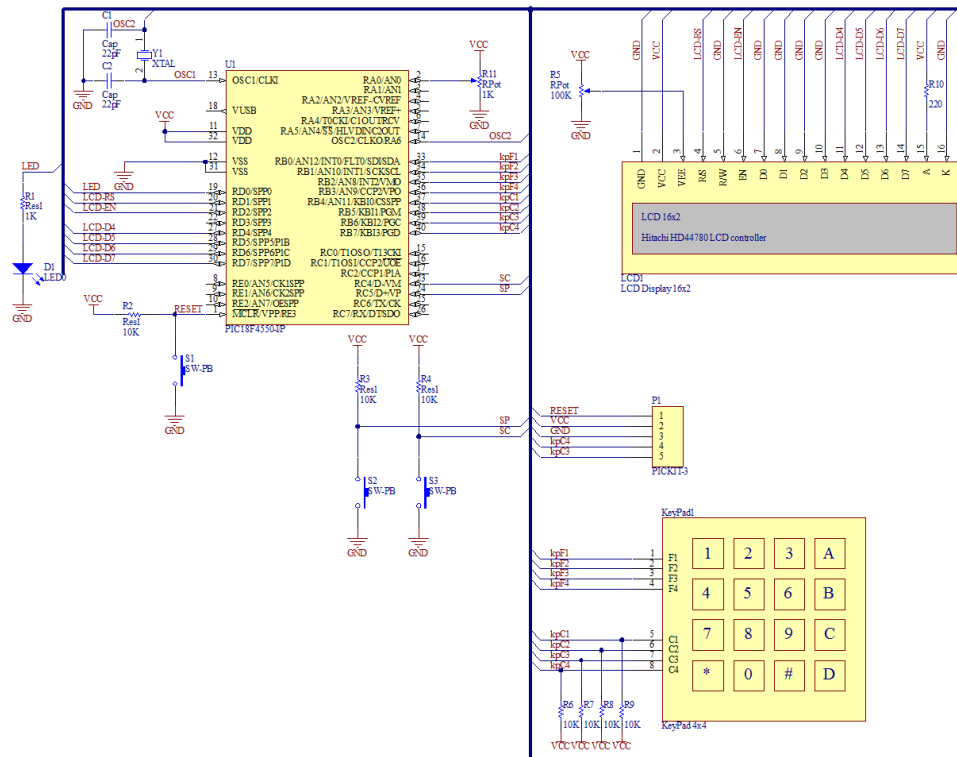


Figura 9: Circuito para manejo de LCD 16×2, teclado matricial y ADC: PIC18F4550

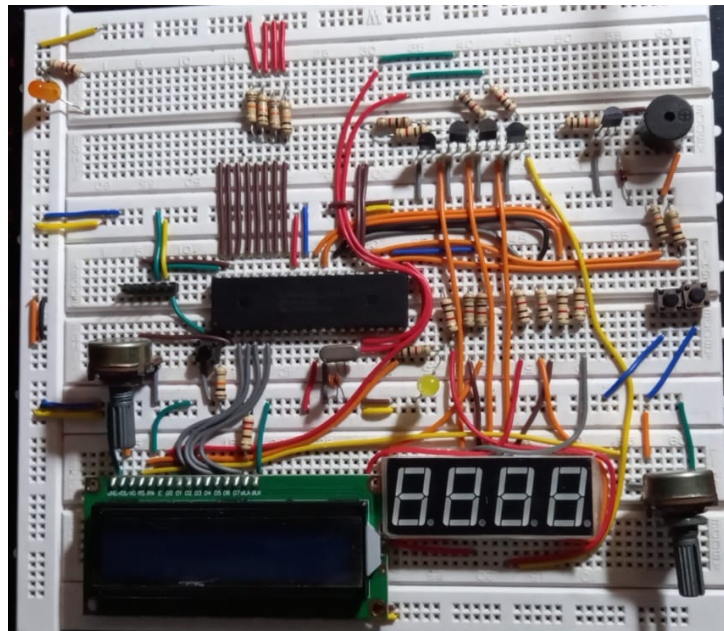


Figura 10: Implementación física del circuito en protoboard

3.1.1. Descripción de Bloques

- **LCD 16×2 (Puerto D + Puerto A):** El LCD se controla en modo 4 bits. Las señales de control RS y EN se conectan a RD1 y RD2, respectivamente. Los datos D4–D7 se conectan a RA1–RA4. El pin RW se conecta directamente a GND (solo escritura).
- **Entrada analógica (RA0/AN0):** El potenciómetro R11 de 100 kΩ divide el voltaje de alimentación, entregando un valor variable de 0 a 5 V al canal AN0 del ADC.
- **Indicador LED (RD0):** LED con resistencia limitadora de 1 kΩ para la rutina de inicialización y modo de velocidad variable según el ADC.
- **Teclado Matricial (Puerto B):** RB4–RB7 como filas (salidas), RB0–RB3 como columnas (entradas) con resistencias pull-up externas de 10 kΩ.

Descripción General del Sistema:

1. **Entrada:** Potenciómetro (AN0) + Teclado matricial 4×4.
2. **Procesamiento:** PIC18F4550 con TMR0 para muestreo periódico del ADC cada 200 ms.
3. **Salida:** LCD 16×2 (valores, menús, mensajes) + LED RD0 (velocidad controlada por ADC).

4. Partes de la Práctica

4.1. Parte 1: Inicialización con LED RD0

4.1.1. Enunciado

Implemente una subrutina de inicialización con el LED en RD0: 10 pulsos en intervalos de 50 ms y luego permanecer encendido hasta que se indique lo contrario.

4.1.2. Descripción

El LED en RD0 produce una secuencia de 10 parpadeos como indicador visual de que el sistema arrancó correctamente. Cada parpadeo alterna el estado del LED con un retardo de 50 ms, completando la secuencia en 500 ms. Al finalizar, el LED queda encendido de forma continua hasta que la lógica del ADC lo controle.

4.1.3. Fragmento de Código — Parte 1

```
1 // =====
2 //             LED PARPADEANDO (Blink)
3 // =====
4 void led_blink(int tiempo_ms){
5     LATDbits.LATD0 = !LATDbits.LATD0; // toggle led
6     DelayXms(tiempo_ms); // delay
7 }
8
9 // main():
10 // subrutina de inicializacion
11 for(char i=0;i<10;i++) led_blink(50);
12 LATDbits.LATD0 = 1; // LED ON permanente
```

Listing 1: Inicialización LED RD0 y función blink — Parte 1

4.2. Parte 2: Visualización de Bienvenida en LCD

4.2.1. Enunciado

Configure el LCD acorde al hardware mostrado. Después de la rutina de inicio marcada por el LED RD0, muestre los siguientes mensajes en el LCD con el cursor apagado: Universidad de Cuenca durante 2 segundos; luego Telecom-2026 en la primera línea y su nombre en la segunda línea durante 2 segundos. Repita este último texto de forma intermitente 3 veces a intervalos de 500 ms.

4.2.2. Descripción

La pantalla LCD se inicializa en modo 4 bits con el controlador HD44780. La comunicación se realiza enviando el byte en dos nibbles (alto y bajo), generando un pulso EN entre cada uno. Los mensajes de bienvenida se muestran de forma secuencial con tiempos controlados por software. La intermitencia de 3 ciclos se logra con un bucle que alterna mostrar y limpiar la pantalla.

4.2.3. Fragmentos de Código — Parte 2

```
1 // =====
2 //             INICIALIZACION LCD
3 // =====
4 void InitLCD(void){
```

```

5 // Inicializacion para LCD 16x2, MODO 4 BITS
6 DelayXms(15); // >=15 ms
7
8 // lcd NO sabe si trabajar en: 8 bits o 4 bits
9 LCD_Comando(0x33); //0011 = 3 HEX
10 LCD_Comando(0x32); //0010 = 2 HEX
11
12 LCD_Comando(0x28); // ES: 4 bits, 2 lineas
13 LCD_Comando(0x0C); // Display ON
14 LCD_Comando(0x06); // Incremento cursor, letra desplaza +1 a la derecha
15 LCD_Comando(0x01); // Clear LCD
16
17 DelayXms(5);
18 }

```

Listing 2: Inicialización LCD en modo 4 bits — Parte 2

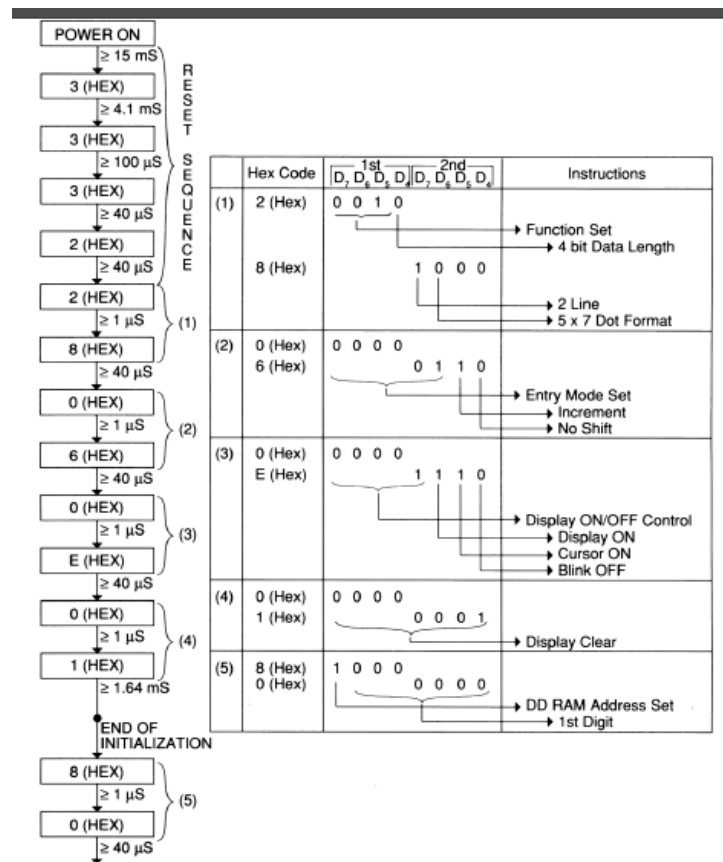


Figura 11: Inicialización para el LCD

```

1 // =====
2 //           ENVIO DE DATOS: MODO 4 BITS (A1,...,A4)
3 // =====
4 void TxData(char dato){
5     /* ejemplo: dato = 0x28 = 00101000
6      * RA1 -> D4; RA2 -> D5; RA3 -> D6; RA4 -> D7
7      */
8     // ----- Nibble Alto (D7 D6 D5 D4) -----
9     LATAbits.LATA1 = (dato & 0x10)>>4; // D4: ...
10    LATAbits.LATA2 = (dato & 0x20)>>5; // D5
11    LATAbits.LATA3 = (dato & 0x40)>>6; // D6
12    LATAbits.LATA4 = (dato & 0x80)>>7; // D7: 0x80 = bit 7: 2^7

```

```

13 BitEN(); // PULSO
14 // ----- Nibble BAJO -----
15 LATAbits.LATA1 = (dato & 0x01); // D0: 0x01 = bit 0: 2^0
16 LATAbits.LATA2 = (dato & 0x02)>>1; // D1: 0x02 = bit 1: 2^1
17 LATAbits.LATA3 = (dato & 0x04)>>2; // D2: ..
18 LATAbits.LATA4 = (dato & 0x08)>>3; // D3
19
20
21 BitEN(); // PULSO
22 Delay1ms();
23 }

```

Listing 3: Envío de datos en modo 4 bits — Parte 2

En la práctica, los mensajes de bienvenida se muestran como parte de la rutina de inicio antes del bucle principal:

```

1 subroutine_init();
2
3 InitLCD(); // Configuración de lcd 16*2, 4 bits
4 // Universidad de Cuenca >> LONGUIUD DE 16 (USA 2 FILAS)
5 LCD_DosLineas("Universidad de", "Cuenca");
6 DelayXms(2000); // Espera 2[seg]
7
8 // punto 4.
9 LCD_DosLineas("Telecom-2026", "Jaime Chiqui");
10 DelayXms(2000); // Espera 2[seg]
11 LCD_Comando(0x01); // Clear LCD
12 DelayXms(1000); // Espera 1[seg]
13
14 for(char i=0;i<3;i++){
15     LCD_DosLineas("Telecom-2026", "Jaime Chiqui");
16     DelayXms(500); // Espera 500[ms]
17     LCD_Comando(0x01); // Clear LCD
18     DelayXms(500); // 500 ms por software
19     // 500 ms LCD ON, 500 ms LCD OFF
20 }
21 while(1);

```

Listing 4: Mensajes de bienvenida y menú — Parte 2

4.3. Parte 3: Visualización del ADC en LCD

4.3.1. Enunciado

Configure un temporizador para generar una interrupción cada 200 ms. Dentro de la ISR se realizará la lectura del ADC y actualización del LCD. Luego de la segunda parte adicionar lo siguiente:

1. Visualice el texto: Universidad de Cuenca, durante 2 segundos y seguidamente limpie la pantalla por 1 segundo.
2. Visualice el Texto: ADC v1.0, durante 2 segundos y seguidamente limpie la pantalla, sin que se muestre nada en la pantalla durante 1 segundo.

En la primera línea visualice: ADC = XXXX donde el número refleja la lectura entre 0 y 1023. Controle la velocidad del LED en RD0 de forma automática según la tabla: 0–256: 25 ms; 256–512: 50 ms; 512–768: 100 ms; 768–1023: 200 ms.

4.3.2. Descripción

El ADC se inicializa en el canal AN0 con justificación derecha y reloj $F_{osc}/64$. El TMR0 genera interrupciones cada 200 ms con prescaler 1:256. En cada interrupción se activa la bandera flagADC, que en el bucle principal dispara la actualización del LCD y el cálculo de velocidad del LED. El valor de 10 bits se descompone en dígitos individuales para mostrarlo en pantalla.

4.3.3. Fragmentos de Código — Parte 3

```
1 // =====
2 //             CONFIGURACION ADC
3 // =====
4 void InitADC(void){
5     //0x0E=1110 : AN0 analogico, AN1...AN12 digitales
6     ADCON1 = 0x0E;
7
8     // ADFM=1 : Right justified
9     // ACQT=001 = 2TAD: A/D acquisition Time select bits
10    // ADCS=110 = Fosc/64: A/D Clock select bits
11    ADCON2 = 0b10001110; // ADCON2: A/D CONTROL REGISTER 2
12
13    // ADCON0: A/D CONTROL REGISTER 0
14    CHS3 = 0; // Canal AN0
15    CHS2 = 0;
16    CHS1 = 0;
17    CHS0 = 0;
18
19    ADON = 1; // ADC ON(ENABLE)
20 }
21 // =====
22 //             LEER ADC 10 BITS
23 // =====
24 void LeerADC(void){
25     Delay1ms(); // tiempo de adquisicion (carga del capacitor)
26
27     // Convertir el voltaje en AN0 a un numero digital
28     GO_DONE = 1; // A/D Conversion Status bit
29     while(GO_DONE); //termina conversion ?GO_DONE = 0
30
31     // Conversion obtenemos adc 10 bits
32     adcValue = ADRESL; //LEE 8 bits bajos
33     temp = ADRESH; // 6 ceros + 2 bits faltantes
34
35     adcValue = adcValue | (temp << 8); // Mover bits 8 posiciones a la izq.
36 }
```

Listing 5: Inicialización del módulo ADC — Parte 3

```
1 // =====
2 //             ISR ALTA PRIORIDAD - TMR0 16 bits
3 // =====
4 void __interrupt(high_priority) InterrupcionHP(void) {
5     if (INTCONbits.TMR0IF) {
6         // TMR0: 200ms (Fosc=48MHz, 16-bit, Prescaler 1:256)
7         // Tcy = 83.33ns; Tick = 21.33us
8         // Conteos = 9375; Carga = 65536 - 9375 = 56161 = 0xDB61
9         TMR0H = 0xDB;
10        TMR0L = 0x61;
11
12        if(!holdADC){
13            LeerADC(); // ADC en la ISR
14        }
15    }
```

```

15     flagADC = 1;
16     INTCONbits.TMR0IF = 0;
17 }
18 }

```

Listing 6: ISR del TMR0 para muestreo cada 200 ms — Parte 3

```

1  // =====
2  //          VISUALIZAR VALOR ADC EN LCD
3  // =====
4  void MostrarADC(void){
5      char miles;
6      char centenas;
7      char decenas;
8      char unidades;
9
10     miles    = adcValue / 1000;
11     centenas = (adcValue % 1000)/100;
12     decenas  = (adcValue % 100)/10;
13     unidades = adcValue % 10;
14
15     LCD_Comando(0x80);
16     LCD_Texto("ADC = ");
17     LCD_Caracter(miles + '0');
18     LCD_Caracter(centenas + '0');
19     LCD_Caracter(decenas + '0');
20     LCD_Caracter(unidades + '0');
21     LCD_Texto(" ");
22 }
23 // =====
24 //          VELOCIDAD LED SEGUN ADC
25 // =====
26 void ActualizarVelocidadLED(void){
27     if(adcValue < 256)
28         ledPeriodo = 25;
29     else if(adcValue < 512)
30         ledPeriodo = 50;
31     else if(adcValue < 768)
32         ledPeriodo = 100;
33     else
34         ledPeriodo = 200;
35 }

```

Listing 7: Visualización del valor ADC en LCD — Parte 3

4.3.4. Tabla de Velocidad del LED según Lectura ADC

Cuadro 2: Control de velocidad del LED RD0 según valor ADC

Rango ADC	Voltaje aprox. (V)	Intervalo LED (ms)
0 – 255	0.0 – 1.25	25 (muy rápido)
256 – 511	1.25 – 2.5	50 (rápido)
512 – 767	2.5 – 3.75	100 (normal)
768 – 1023	3.75 – 5.0	200 (lento)

4.4. Parte 4: Funcionalidad Propia — Menú Interactivo con Teclado

4.4.1. Enunciado

Cada estudiante debe agregar una funcionalidad propia que involucre el uso del teclado matricial.

4.4.2. Descripción

Se implementó un sistema de menú interactivo con 3 modos de operación seleccionables desde el teclado matricial, demostrando la integración completa entre ADC, LCD y teclado:

- **Tecla A → MODO_ADC_AUTO:** Muestra el valor ADC en tiempo real en la línea 1 del LCD. El LED parpadea a la velocidad determinada por el potenciómetro (*Parte 3 de esta practica*).
- **Tecla B → MODO_LED_MANUAL:** El usuario selecciona la velocidad del LED con las teclas 1–4 (25, 50, 100, 200 ms), respectivamente. La línea 2 del LCD muestra una barra de progreso proporcional al valor ADC.
- **Tecla C → MODO_JUEGO:** Juego de adivinanza del valor ADC. El usuario ingresa 4 dígitos con el teclado numérico y presiona D para comparar. Si la diferencia con el ADC real es ≤ 20 , suma un punto. El LCD muestra el puntaje acumulado.
- **Tecla * → Reset:** Muestra REINICIANDO y regresa al menú principal.
- **Tecla # → Hold ADC:** Congela o libera la lectura del ADC (holdADC).

4.4.3. Fragmentos de Código — Parte 4

```
1 // =====
2 //           MODOS DE FUNCIONAMIENTO
3 // =====
4 #define MODO_ADC_AUTO      0
5 #define MODO_LED_MANUAL    1
6 #define MODO_JUEGO        2
7 #define MODO_MENU         3
8
9 // =====
10 //           MENU PRINCIPAL
11 // =====
12 void MostrarMenuScroll(void){
13     LCD_Comando(0x01);
14     LCD_Texto("A:ADC B:LED");
15     LCD_Linear2();
16     LCD_Texto("C:JUEGO *,#:RST");
17 }
```

Listing 8: Definición de modos y menú principal — Parte 4

```
1 // =====
2 //           ADC COMO BARRA DE PROGRESO
3 // =====
4 void MostrarBarraADC(void){
5     unsigned char barras;
6     barras = (adcValue * 16)/1023;
7     LCD_Linear2();
8     for(char i=0;i<16;i++){
9         if(i < barras)
10             LCD_Caracter(255); // Caracter de bloque solido
11         else
12             LCD_Caracter(' ');
13     }
14 }
15 // =====
16 //           VELOCIDAD LED MANUAL SEGUN TECLA
17 // =====
18 void AjustarLEDManual(char tecla){
```

```

19     switch(tecla){
20         case 1: ledManual = 25; break;
21         case 2: ledManual = 50; break;
22         case 3: ledManual = 100; break;
23         case 4: ledManual = 200; break;
24     }
25 }

```

Listing 9: Barra de progreso ADC y velocidad LED manual — Parte 4

```

1  // =====
2  //          MOSTRAR SCORE
3  // =====
4  void MostrarScore(char correcto){
5      LCD_Comando(0x01);
6      if(correcto)
7          LCD_Texto("CORRECTO!");
8      else
9          LCD_Texto("INCORRECTO");
10     LCD_Linea2();
11     LCD_Texto("PTS:");
12     LCD_Caracter((puntaje/10)+'0');
13     LCD_Caracter((puntaje%10)+'0');
14 }
15
16 // En el bucle principal (tecla D confirma intento):
17 if(modo == MODO_JUEGO && tecla == 13){
18     int diferencia;
19     diferencia = adcValue - numeroIngresado;
20     if(diferencia < 0) diferencia = -diferencia;
21
22     if(diferencia <= 20){
23         puntaje++;
24         MostrarScore(1);    // Correcto
25     }
26     else{
27         MostrarScore(0);    // Incorrecto
28     }
29     DelayXms(2000);
30     numeroIngresado = 0;
31     digitosIngresados = 0;
32 }

```

Listing 10: Lógica del juego de adivinanza ADC — Parte 4

```

1  // MODO SEGUN TECLADO
2  if(tecla == 10) modo = MODO_ADC_AUTO;    // TECLA A
3  if(tecla == 11) modo = MODO_LED_MANUAL;  // TECLA B
4  if(tecla == 12){                          // TECLA C
5      modo = MODO_JUEGO;
6      numeroIngresado = 0;
7      digitosIngresados = 0;
8  }
9  if(tecla == 14){    // * -> Reset
10     ResetLCD();
11     modo = MODO_MENU;
12 }
13 if(tecla == 15){    // # -> Hold ADC
14     holdADC = !holdADC;
15     LCD_Comando(0x01);
16     if(holdADC)
17         LCD_Texto("ADC HOLD");
18     else

```



```

19 LCD_Texto("ADC RUN");
20 DelayXms(1000);
21 }

```

Listing 11: Selección de modo y manejo de teclas especiales — Parte 4

4.4.4. Máquina de Estados — Menú Interactivo

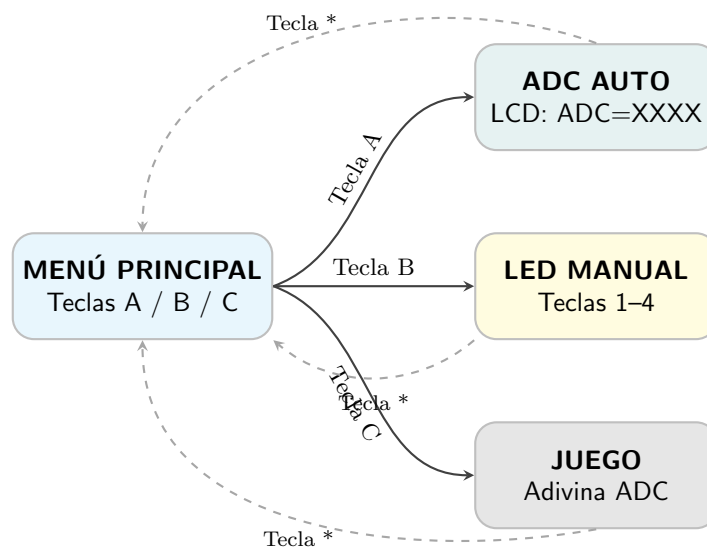


Figura 12: Máquina de estados del menú interactivo — Parte 4

5. Pruebas y Verificaciones

5.1. Parte 1: Inicialización LED

Cuadro 3: Pruebas de la rutina de inicialización del LED RD0

Acción	Resultado Esperado
Encender el sistema	LED RD0 parpadea exactamente 10 veces a intervalos de 50 ms
Al finalizar los 10 pulsos	LED RD0 queda encendido permanentemente

5.2. Parte 2: LCD y Mensajes de Bienvenida

Cuadro 4: Pruebas de inicialización y visualización de mensajes en LCD

Acción	Resultado Esperado
Inicialización del LCD	LCD configurado en modo 4 bits, 2 líneas y cursor apagado
Finalización de la rutina de inicio (RD0)	Se inicia la secuencia de mensajes de bienvenida en el LCD
Primer mensaje mostrado	LCD visualiza “Universidad de” en la primera línea y “Cuenca” en la segunda durante 2 segundos
Limpieza de pantalla	Después del primer mensaje, la pantalla se limpia correctamente
Segundo mensaje mostrado	LCD visualiza “Telecom-2026” en la primera línea y “Jaime Chiqui” en la segunda durante 2 segundos
Pantalla en blanco	Tras el segundo mensaje, el LCD permanece limpio durante 1 segundo
Parpadeo de mensaje	El texto “Telecom-2026” y “Jaime Chiqui” aparece y desaparece 3 veces
Temporización de parpadeo	Cada ciclo de encendido y apagado se realiza con intervalos de 500 ms controlados por software
Alineación de texto	Los caracteres se muestran correctamente en ambas líneas sin desplazamientos ni errores

5.3. Parte 3: Lectura y Visualización del ADC

Cuadro 5: Pruebas del módulo ADC y actualización en LCD

Acción	Resultado Esperado
Potenciómetro al mínimo (GND)	LCD muestra ADC = 0000, LED muy rápido (25 ms)
Potenciómetro al 50 % (≈ 2.5 V)	LCD muestra ADC = 0511, LED normal (100 ms)
Potenciómetro al máximo (5 V)	LCD muestra ADC = 1023, LED lento (200 ms)
Giro continuo del potenciómetro	Valor actualiza cada 200 ms en tiempo real

5.4. Parte 4: Menú Interactivo con Teclado

Cuadro 6: Pruebas del menú interactivo y juego de adivinanza

Acción	Resultado Esperado
Tecla B, luego tecla 1	LED parpadea a 25 ms independientemente del potenciómetro
Tecla B, luego tecla 4	LED parpadea a 200 ms
Barra progreso en modo B	Barra crece proporcionalmente al girar potenciómetro
Tecla C, ingresar número cercano al ADC (± 20)	LCD muestra CORRECTO! PTS: XX+1
Tecla C, ingresar número lejano al ADC (> 20)	LCD muestra INCORRECTO PTS: XX
Tecla * desde cualquier modo	LCD muestra REINICIANDO y regresa al menú

6. Códigos Fuente y Repositorio

Los códigos fuente desarrollados en esta práctica están implementados en C/C++ con MPLAB X IDE v5.50, y se encuentran junto con los videos de funcionamiento en el repositorio de GitHub:

- **Repositorio principal:**
<https://github.com/jaimitochiquic/Micros>
- **Códigos fuente (Práctica 5):**
<https://github.com/jaimitochiquic/Micros/tree/main/Codigos/Pr5>
- **Video Partes 1, 2 y 3 — LED, LCD y ADC:**
https://github.com/jaimitochiquic/Micros/blob/main/Videos/P5/parte1_2_3.mp4
- **Video Parte 4 — Menú interactivo con teclado:**
<https://github.com/jaimitochiquic/Micros/blob/main/Videos/P5/parte4.mp4>

7. Conclusiones y Recomendaciones

7.1. Conclusiones

Con base en la resolución de esta práctica se concluye que:

- La comunicación con el LCD 16*2 en modo 4 bits permite controlar hasta 32 caracteres alfanuméricos usando únicamente 6 pines del microcontrolador (RS, EN, D4–D7), en contraste con los 4 displays de 7 segmentos de la práctica anterior que requerían 11 pines y únicamente podían mostrar dígitos y algunas letras limitadas. El LCD ofrece mayor flexibilidad de visualización con menor uso de hardware.
- Se comprobó que una misma pantalla LCD puede utilizarse para visualizar diferentes tipos de información (mensajes de bienvenida, lecturas ADC, menús y resultados del juego) mediante una adecuada gestión temporal del software, optimizando el uso de los recursos disponibles del microcontrolador.
- La transmisión en dos nibbles exige un orden estricto (alto primero, luego bajo) y un pulso EN entre cada uno. Omitir el tiempo de estabilización mínimo de 15 ms en la inicialización o los delays entre comandos provoca que el HD44780 no interprete correctamente los datos, resultando en caracteres corruptos o pantalla en blanco.
- El módulo ADC del PIC18F4550 entrega 10 bits de resolución, permitiendo distinguir $2^{10} = 1024$ niveles entre 0 y 5 V, es decir una resolución de $\frac{5}{1023} \approx 4,9 \text{ mV}$ por paso.
- Realizar la lectura del ADC dentro de la ISR del TMR0 cada 200 ms garantiza un muestreo periódico y determinista, independiente del tiempo que el bucle principal dedique a otras tareas como la gestión del menú o la escritura en el LCD.
- El diseño orientado a modos (MODO_MENU, MODO_ADC_AUTO, MODO_LED_MANUAL, MODO_JUEGO) demostró ser una arquitectura escalable: añadir un nuevo modo requiere únicamente una nueva constante, una tecla de activación y una función de visualización, sin modificar la estructura del bucle principal ni la ISR.

Referencias

- [1] Microchip Technology Inc., *PIC18F2455/2550/4455/4550 Data Sheet*, Document DS39632E, 2024. [En línea]. Disponible en: <https://ww1.microchip.com/downloads/en/devicedoc/39632e.pdf>
- [2] Hitachi Semiconductor, *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver*, Datasheet ADE-207-272(Z), 1999. [En línea]. Disponible en: <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>
- [3] Microchip Technology Inc., *MPLAB® XC8 C Compiler User's Guide*, Document DS50002053, 2023.

8. Preguntas

1. Indique cómo modificaría el hardware de esta práctica para leer un transductor de humedad que tiene una salida lineal de 0 a 12 voltios en lugar del potenciómetro:

a) Diagrama electrónico:

El PIC18F4550 acepta un máximo de 5 V en sus pines analógicos. Para adaptar la señal de 0–12 V se requiere un divisor resistivo:

Figura 13: Divisor resistivo para adaptar 0–12 V a 0–5 V en la entrada AN0

Cálculo del divisor resistivo para adaptar la salida del sensor (0–12 V) al rango permitido por el ADC del PIC18F4550 (0–5 V):

$$V_{out} = V_{in} \left(\frac{R_2}{R_1 + R_2} \right)$$

Considerando la condición de diseño:

$$V_{out,max} = 5 \text{ V} \quad V_{in,max} = 12 \text{ V}$$

se obtiene:

$$5 = 12 \left(\frac{R_2}{R_1 + R_2} \right)$$

Despejando:

$$5R_1 + 5R_2 = 12R_2$$

$$R_2 = \frac{5}{7}R_1$$

Seleccionando un valor comercial de:

$$R_1 = 14 \text{ k}\Omega$$

se obtiene:

$$R_2 = \frac{5}{7}(14 \text{ k}\Omega) = 10 \text{ k}\Omega$$

Por lo tanto, se utilizan los valores comerciales:

$$R_1 = 14 \text{ k}\Omega \quad R_2 = 10 \text{ k}\Omega$$

Verificación:

$$V_{out} = 12 \left(\frac{10}{14 + 10} \right)$$

$$V_{out} = 12 \left(\frac{10}{24} \right) = 5 V$$

Por lo tanto, cuando el sensor entrega su valor máximo de 12 V, la entrada analógica del microcontrolador recibe exactamente 5 V, evitando sobrepasar el rango permitido por el ADC.

b) Cambios en el software:

El ADC sigue leyendo 0–1023, pero ahora representa 0–12 V. Para mostrar la humedad en porcentaje (asumiendo que 0 V = 0 % HR y 12 V = 100 % HR):

```

1 // Conversion: ADC representa 0-12V = 0-100% HR
2 // V_real = adcValue * 12.0 / 1023.0
3 // Humedad (%) = adcValue * 100 / 1023
4 unsigned int humedad = (unsigned long)adcValue * 100 / 1023;
5
6 char decenas = humedad / 10;
7 char unidades = humedad % 10;
8
9 LCD_Comando(0x80);
10 LCD_Texto("HR = ");
11 LCD_Caracter(decenas + '0');
12 LCD_Caracter(unidades + '0');
13 LCD_Texto(" % ");

```

Listing 12: Conversión ADC a porcentaje de humedad

2. ¿Cómo se realizaría la lectura de dos diferentes tipos de transductores usando el sistema base provisto en esta práctica?

Respuesta:

El PIC18F4550 dispone de hasta 13 canales analógicos (AN0–AN12), como se aprecia en la Figura 6. Para leer dos transductores simultáneamente se procede de la siguiente manera:

- a) **Configurar ADCON1** para habilitar dos canales analógicos. Por ejemplo, ADCON1 = 0x0D habilita AN0 y AN1 como analógicos y el resto como digitales.
- b) **Multiplexar el canal** cambiando los bits CHS3:CHS0 de ADCON0 antes de cada conversión:

```

1 // Leer AN0 (Transductor 1)
2 ADCON0bits.CHS = 0; // Canal AN0
3 Delay1ms(); // Tiempo adquisicion
4 GO_DONE = 1;
5 while(GO_DONE);
6 adc_canal0 = ADRESL | ((unsigned int)ADRESH << 8);
7
8 // Leer AN1 (Transductor 2)
9 ADCON0bits.CHS = 1; // Canal AN1
10 Delay1ms();
11 GO_DONE = 1;
12 while(GO_DONE);
13 adc_canal1 = ADRESL | ((unsigned int)ADRESH << 8);

```

Listing 13: Lectura multiplexada de dos canales ADC

- c) Cada lectura se procesa de forma independiente, mostrando cada resultado en una línea del LCD o alternando la visualización con una tecla del menú.

3. Nombre las ventajas que identificó al utilizar un LCD 16×2 en lugar del conjunto de displays de siete segmentos.

Respuesta:

- **Mayor capacidad de información:** El LCD muestra hasta 32 caracteres alfanuméricos por pantalla, incluyendo letras, símbolos y mensajes completos. Los displays de 7 segmentos de la práctica anterior solo pueden mostrar 4 dígitos y algunas letras limitadas (H, r, E, L, F, n, etc.).
- **Menor uso de pines:** El LCD en modo 4 bits requiere 6 pines (RS, EN, D4–D7). Los 4 displays de 7 segmentos multiplexados requerían 11 pines (7 segmentos + 4 transistores de control), además de resistencias individuales por segmento.
- **Menor carga de la CPU:** Con el LCD no se requiere una ISR dedicada al refresco visual. El PIC18F4550 escribe el dato y el HD44780 lo mantiene automáticamente, liberando tiempo de cómputo que en la práctica anterior debía dedicarse a la multiplexación cada 2.5–5 ms.
- **Facilidad de programación para texto:** Mostrar la cadena ADC = 0578 requiere una sola llamada a LCD_Texto(). Con displays de 7 segmentos, cada dígito requiere convertir el número a su código binario de segmentos y luego multiplexarlo individualmente.
- **Versatilidad:** El LCD puede mostrar menús, mensajes de error, estado del sistema y valores con unidades (mV, etc.), lo que resulta imposible con 7 segmentos.
- Menor cantidad de líneas de control en comparación con múltiples displays de siete segmentos.